

Laboratório de Sistemas Digitais (LSD)



<https://www.technolution.eu/en/about-us/publications/417-new-developments-make-fpgas-accessible-for-everybody.html>

Linguagem de descrição de Hardware (HDL)

Linguagem que procura descrever a configuração de hardware de um circuito e não a lista de instruções que o circuito deve executar.

HDL – Linguagem de Descrição de Hardware. Permite que linguagens rigidamente definidas representem circuitos lógicos.

HDL -Hardware Description Language

Verilog - Verilog é uma das duas maiores HDL usadas por projetista de hardware na indústria e no meio acadêmico .

Foi desenvolvida entre 1984 e 1985 por Philip Moorby que necessitava de um meio simples, intuitivo e efetivo de descrever circuitos digitais para modelagem, simulação e análise.

Em 1985 – tornou-se propriedade da **Gateway** Design System Corporation. Posteriormente foi adquirida pela **Cadence** Design Systems Inc.,

Em **1990**, a linguagem foi aberta para domínio público com a expectativa que com isso aumentasse rapidamente. O padrão atual da linguagem Verilog é o IEEE1364 de dezembro de 1995.

Introdução à Verilog HDL

VERILOG

- É um pouco mais utilizada na indústria do que outras
- É baseada em C, ao contrário de VHDL, que é baseada em Ada
 - Quem está familiarizado com C achará mais fácil acompanhar os fundamentos da Verilog
 - Quem conhece VHDL deverá achar os conceitos simples, desde que já conheçam um pouco da sintaxe da linguagem C
- Suporta projeção, verificação e implementação de projetos analógicos, digitais e híbridos em vários níveis de abstração

ATENÇÃO

Primeiro caractere de não pode ser um número.

Cuidado com **CARACTERES ESPECIAIS** e **espaços** no nome da pasta, isso pode ser um problema 😞

Keywords

always and assign attribute begin buf bufif0 bufif1 case casex casez cmos deassign default defparam disable edge else end endattribute endcase endfunction	endmodule endprimitive endspecify endtable endtask event for force forever fork function highz0 highz1 if ifnone initial inout input integer join medium module	large macromodule nand negedge nmos nor not notif0 notif1 or output parameter pmos posedge primitive pull0 pull1 pulldown pullup rcmos real realtime	reg release repeat rnmos rpmos rtran rtranif0 rtranif1 scalared signed small specify specparam strength strong0 strong1 supply0 supply1 table task time tran	tranif0 tranif1 tri tri0 tri1 triand trior triereg unsigned vectored wait wand weak0 weak1 while wire wor xnor xor
---	--	---	---	---

CONVENÇÕES

Comentários

// Comentários de uma linha só

/ outra forma de comentário de uma linha */*

/ inicio de comentário com múltiplas linhas*

Todo o texto é ignorado

Terminada com a linha abaixo

**/*

MÓDULO/ENTIDADE

Um projeto é formado a partir da construção de blocos, e em Verilog cada bloco é chamado de **módulo (module)**.

- Descreve a funcionalidade do circuito
- Define terminais (pinos, portas) de entrada e saída

Deve-se dar um nome ao **PROJETO** que represente o circuito descrito.



Como iniciar um projeto?



Atenção com o
uso de
caracteres

- Criar um projeto e atribuir um nome.
- Criar o código de HDL do *module main* (principal) que descreve o circuito – **DEVENDO** este ter o mesmo nome do PROJETO principal.
 - Definir as entradas e saídas - *ports* (interfaces externas)
 - Definir sinais internos
 - Definir o corpo do circuito (funcionamento)

O **Verilog** é uma linguagem ***CASE SENSITIVE***, isto é, ela diferencia as letras maiúsculas das minúsculas. E, todas as palavras-chaves devem ser escritas em minúsculas.

Dados externos - declarados nos *ports*

ports (interface externa) são os dados de entrada e saída que o usuário terá contato:

- teclado, mouse (entrada)
- Vídeo, som (saída)

- chaves, botões (*push buttons*)
- Leds, Display de 7-segmentos, LCD

Dados externos - declarados nos *ports*

Os *ports* são caracterizados pelo **tipo e direção** do sinal. Por tipo, **entende-se sinal simples (1 bit)** ou **barramento (8bits)**.

Em termos de **direção**, pode-se ter:

- **INPUT**– dados são lidos pelo módulo.
- **OUTPUT** – dados são externados pelo módulo.
- **INOUT**– permite que dados entrem ou saiam do módulo e são chamados de bidirecionais.

Especificação do *port*

- Colocar a lista de *ports* entre parênteses após o nome do *module*;
- Especificar a **direção**, o **tipo** e o **nome** do *port* no cabeçalho do *module*.

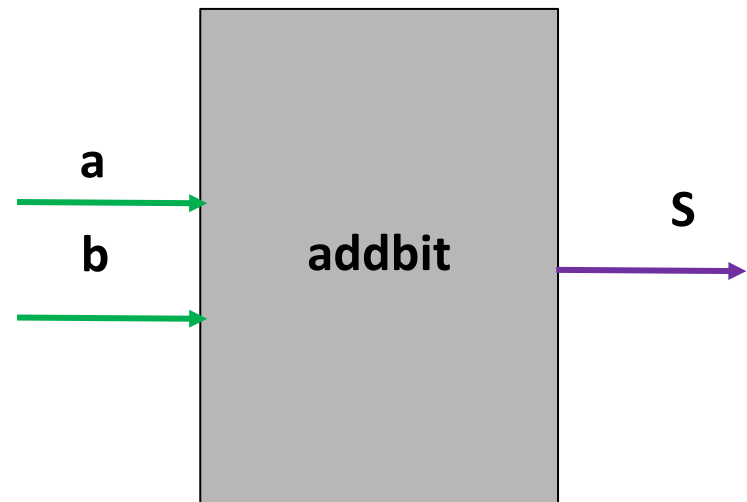
Exemplos

<direção>	<tipo da porta>	<nome da porta>
input	[7:0]	a,b; // entrada de 8 bits
inout	[7:0]	clk,clr; // entrada/saída de 8 bits
output	[15:0]	out; // saída de 16 bits

DECLARAÇÕES DE *PORTS* (PORTAS)

EXEMPLO

```
module addbit (a,b,S);  
  //Declaração de Portas  
  input a;  
  input b;  
  output S;  
  
  // descrição do Comportamento  
  
endmodule
```



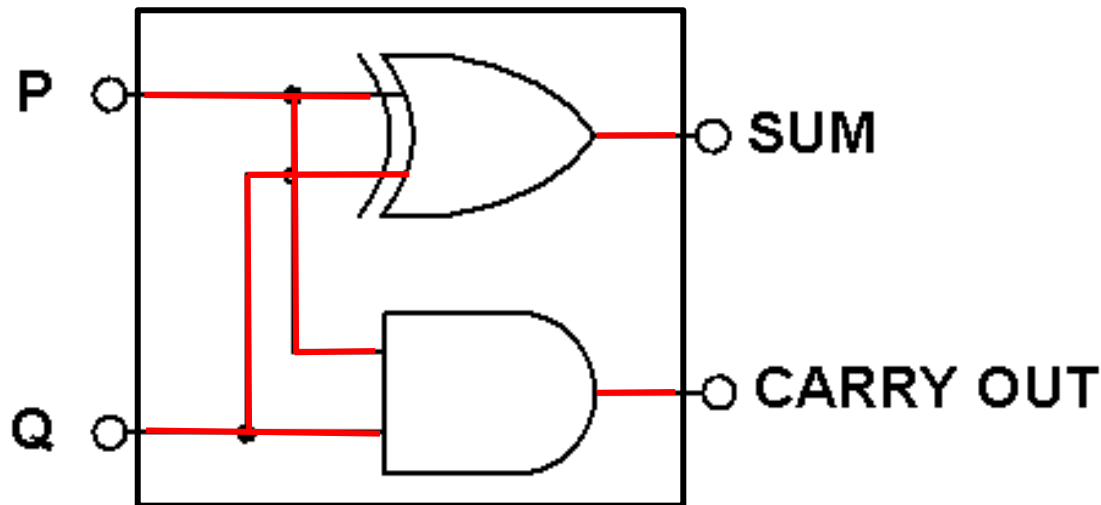
Tipos de dados físicos

Nets – tipo utilizado para conectar componentes

Exemplo:

wire [3:0] **Sa**; // conexão de 4 bits entre componentes e não tem capacidade de armazenamento

Fios são frequentemente usados para conectar componentes.



Tipos de dados físicos

Register – representa elementos capazes de armazenar dados temporariamente

Exemplo:

reg A;

reg [0:7] B;

// [7:0] B contém 8 bits (menos significativo à direita)

// [0:7] B contém 8 bits (mais significativo à direita)

TIPOS DE DADOS ABSTRATOS

Integer

```
integer count;    // inteiro simples de 32 bits com sinal  
integer k [64:1]; //vetor de 64 inteiros
```

Real

```
real a;
```

Time

```
time x, y; //Duas variáveis time de 64 bits
```

Realtime

```
realtime z;
```

Especificações de números

- Números com comprimento definido:

<comprimento>'<base numérica><número>

- Números sem comprimento definido:

'<base numérica><número>

Os números sem a definição da *<base numérica>* são entendidos como decimal.

Formato dos Números:

Bases numéricas:

b – Binário

d – Decimal

o – Octal

h – Hexadecimal

<comprimento>'<base numérica><número>

Especificações de números

Exemplos:

Decimal ('d) -> 16'd255 = número 255, decimal, armazenado com 16 bits

Hexadecimal ('h) -> 8'h9a = número 9a, hexadecimal, armazenado com 8 bits

Binário ('b) -> 'b1010 = número 10, binário, armazenado com 32 bits

Octal ('o) -> 'o21 = número 21, octal, armazenado com 32bits

Números Negativos

- 8'd3 = número 3, negativo, armazenado com 8 bits

Exemplos de números

213 // número decimal

8'h5A // número hexadecimal com 8 bits

9'o321 // número octal com 9 bits

4'b11 // número binário de 4-bits: 0011

3'b10x // número binário de 3-bits com último bit desconhecido

5'd3 // número decimal de 5-bits

Operadores

Operadores unário, binário.

- Operadores unários - precedem o operando.

$$a = \sim b$$

*$\sim$ é um operador unário. **b** é o operando*

- Os operadores binários aparecem entre os operandos.

$$a = b \& c$$

*$\&$ é um operador binário. **b** e **c** são operandos*

Operadores

Unário	
&	AND
 	OR
^	XOR
~&	NAND
~ 	NOR
~^	XNOR

Exemplos: operadores unários

// B: 4'b1110

A = &B;

// Resultado A: 0

// B: 4'b1110

A = |B;

// Resultado A: 1

Operadores – aritméticos binários

Operadores aritméticos	
-	subtração
*	multiplicação
/	divisão
%	Módulo ou resto da divisão
+	adição

OBS: Divisão por 0 resulta em um valor desconhecido

Operador Unário	
-	Muda o sinal do operador

Operadores

Relacionais	
>	maior
>=	Maior ou igual
<	Menor que
<=	Menor ou igual
==	igual
!=	diferente

Operadores

Lógicos

!	negação
&&	Conjunção
	Disjunção
?	condicional

// A: true

if (!A) ...

// Resultado: false

if A && B

Operadores

Entre bits	
$\sim$	negação
$\&$	conjunção
$ $	OR
$\wedge$	XOR
$\sim\&$	NAND
$\sim $	NOR
$\sim\wedge$	XNOR
$\wedge\sim$	XNOR

Outros operadores

{a,b}	Concatenação
<<	Deslocar bits a esquerda
>>	Deslocar bits a direita

Espaços de bits vazios preenche com zeros

Concatenação

// **B: 4'b110** **C: 4'b10** ou B: 4'b0110 C: 4'b0010

A = {B, C};

D = {2{B}};

E = {B, C[1:0]};

// Resultado **A: 8'b01100010**

// Resultado **D: 8'b01100110**

// Resultado **E: 6'b011010**

```
Deslocamento de bits a esquerda  
// A: 8'b1110 1101  
  A << 2;  
// Resultado A: 8'b1011 0100
```

```
CONDICIONAL  
// A: 1'b1   B: 3'b111   C: 3'b001  
  
  A == 1? B : C;  
  
// Resultado A: 3'b111
```

Comando Redução

Resume uma quantidade de bits em uma única operação.

assign and4 = &a;

Equivale a:

assign and4 = a[3] & a[2] & a[1] & a[0];

Possível também para: & (AND),
 | (OR),
 ^ (XOR)

Comandos lógicos

Os blocos **sem memória** são chamados **combinacionais**;

- a saída de um bloco combinacional depende apenas da(s) entrada(s) atual(is):
- Como um bloco lógico **combinacional** não contém memória, ele pode ser especificado completamente definindo os valores das saídas para cada conjunto de valores de entrada possíveis;
- Em Verilog, descrevemos a lógica **combinacional** sempre que possível usando o comando de atribuição **assign**
 - $E = A \wedge B \wedge C$ (OR EXCLUSIVO)
 - $D = A \mid B \mid C$
 - $F = A \& B \& C$

Comandos Lógicos

- Nos **blocos com memória**, as saídas podem depender das entradas e do valor armazenado na memória; esses circuitos são chamados de **sequenciais**.

Comando de atribuição **assign**

A forma de se **atribuir** um valor a uma variável é usando o comando ***assign***, e o valor atribuído é proveniente de uma operação lógica:

```
assign S = A & B;
```

Caso haja atraso, este deve ficar imediatamente após o comando ***assign***.

O receptor do resultado pode ser um ***wire***, mas não um ***reg***.

Comando de atribuição **assign**

- Como inserir Atraso:

Exemplo:

```
assign #2 Y = a ^ b;
```

“#2” indica 2 unidades de tempo

- Sem especificação de atraso: 0 (default)

Comando de atribuição **assign**

Declarar e atribuir valores a X

// declaração explícita

```
module Gama (Z, X, Y):  
  output Z;  
  input X, Y;  
  wire A;  
  assign A = X & Y;  
endmodule
```

// declaração implícita

```
module Gama (Z, X, Y):  
  output Z;  
  input X, Y;  
  wire A = X & Y;  
endmodule
```

Comando de atribuição **assign**

Declarar e atribuir **valores** a X

// declaração explícita

```
module Gama (Z, X, Y):  
    output Z;  
    input X, Y;  
    wire A;  
    assign A = X & Y;  
endmodule
```

// declaração implícita

```
module Gama (Z, X, Y):  
    output Z;  
    input X, Y;  
    wire A = X & Y;  
endmodule
```

Atenção

Declaração e atribuição de uma só vez a Z NÃO SE PODE ATRIBUIR MAIS

Blocos de declarações de procedimentos

Existem duas maneiras:

- **initial**: Executa uma única vez no início da simulação, NÃO sintetizável
- **always**: Executa repetidamente, sintetizável

initial

Inicializa as variáveis.

```
initial
```

```
begin
```

```
    a = 0;
```

```
    b = 0;
```

```
end
```

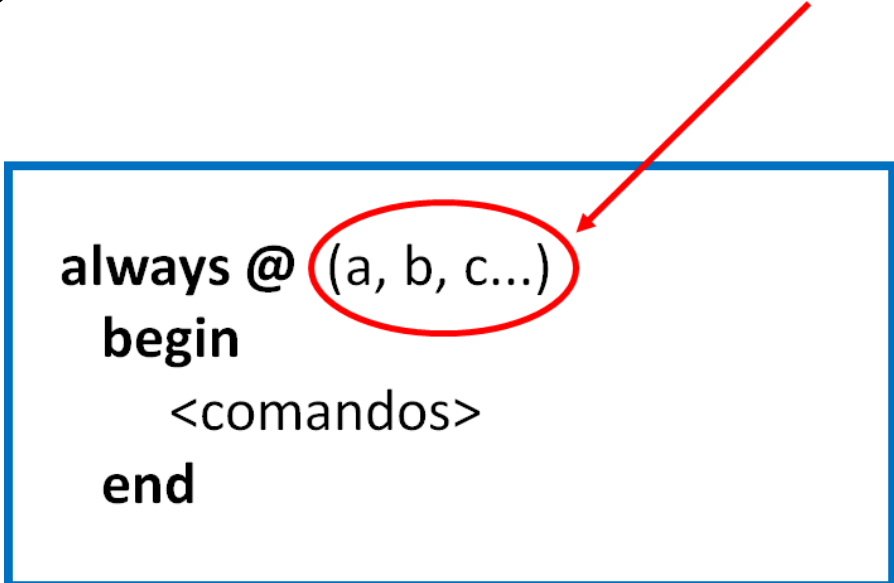
Comandos Lógicos

- Circuitos **combinacionais** podem ser modelados usando comandos **assign** e **always**;
- Circuitos **sequenciais** só podem ser modelados com comando **always**.

always

Funciona como “loop de comandos”;

- **always** recebe como parâmetros uma lista de sensibilidade, ou seja, uma lista que diz quando o bloco de código será executado



```
always @ (a, b, c...)
begin
    <comandos>
end
```

The diagram illustrates the syntax of the 'always' statement. A red arrow points from the text 'lista de sensibilidade' in the list above to the sensitivity list '(a, b, c...)' in the code block below. The sensitivity list is circled in red, and the entire code block is enclosed in a blue border.

always

- Quando o parâmetro na lista de sensibilidade for (*)
 - **always** vai ser executado quando qualquer variável (porta) de entrada realizar qualquer transição

```
always @ (*)  
begin  
    <comandos>  
end
```

always

- Se tiver mais que um comando **always**, todos os comandos **always** serão executados em paralelo (concorrentes);
- **internamente** a um comando **always**, os comandos são executados em sequência;
- Não podem ser aninhados;
- A variável que recebe a expressão deve ser um **reg**



Ver ESTRUTURA
HIERÁRQUICA

EXEMPLOS

```
module mux_21 (Y,I0,I1,sel);  
    output reg Y;  
    input I0, I1, sel;  
  
    always@(*)  
    begin  
        Y = ((~sel) & I0) | (sel & (I1));  
    end  
endmodule
```



Declarações de comportamento

- Uso de instruções alto nível da linguagem
- Sempre ocorrem dentro de um **always**
 - If-else
 - Case
 - Loop

IF-ELSE

Formato

```
if (<condição>
    comando;
else if (<condição>)
    comando;
else
    comando;
```

```
if (reset)
    Q = 0;
else
    Q = D;
```

```
always @(I0,I1, sel)
begin

    if (sel == 0)
        saida = I0;
    else
        saida = I1;

end
```

CASE

```
case (sinal)
```

```
    condição : comando; ...
```

```
    condição : comando;
```

```
endcase
```

```
case (X)
```

```
    2'b00 : Y = A+B;
```

```
    2'b01 : Y = A-B;
```

```
    2'b10 : Y = A/B;
```

```
endcase
```

```
always @(sel)
```

```
begin
```

```
    case (sel)
```

```
        1'b0 : saida = I0;
```

```
        1'b1 : saida = I1;
```

```
    endcase;
```

```
end
```

CASE

Instrução condicional CASE – com mais de uma saída para uma única condição

```
always @(sel)
    begin

        case (sel)
            1'b0 : begin
                    s = l0;
                    w = l1;
                end
            1'b1 : saida = l1;
        endcase;
    end
```

LOOP

- Forever
- Repeat
- While
- For

FOREVER E REPEAT

Repeat

O comando REPEAT é executado a quantidade de vezes que está definido no parenteses

```
repeat (3)
begin
  count = count + i;
  i = i + 1;
end
```

FOREVER E REPEAT

Forever and Repeat Loops

- **forever** loop - executes continually

```
initial
begin
    clk = 0;
    forever #25 clk = ~clk;
end
```

Clock with period
of 50 time units

- **repeat** loop - executes a fixed number of times

```
if (rotate == 1)
    repeat (8)
        begin
            tmp = data[15];
            data = {data << 1, tmp};
        end
```

Repeats a rotate
operation 8 times

WHILE

```
while(i < 10)
begin
    count = count + 1;
    i = i + 1;
end
```

WHILE

While Loop

- **while** loop - executes if expression is true

```
initial
begin
    count = 0;
    while (count < 101)
        begin
            $display ("Count = %d", count);
            count = count + 1;
        end
    end
end
```

Counts from 0 to 100
Exits loop at count 101

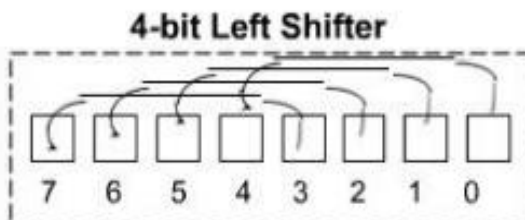
FOR

```
for( i = 0; i < 10; i = i + 1)
  begin
    count = count + 1;
  end
```

FOR

For Loop

- Executes once at the start of the loop and then executes if expression is true

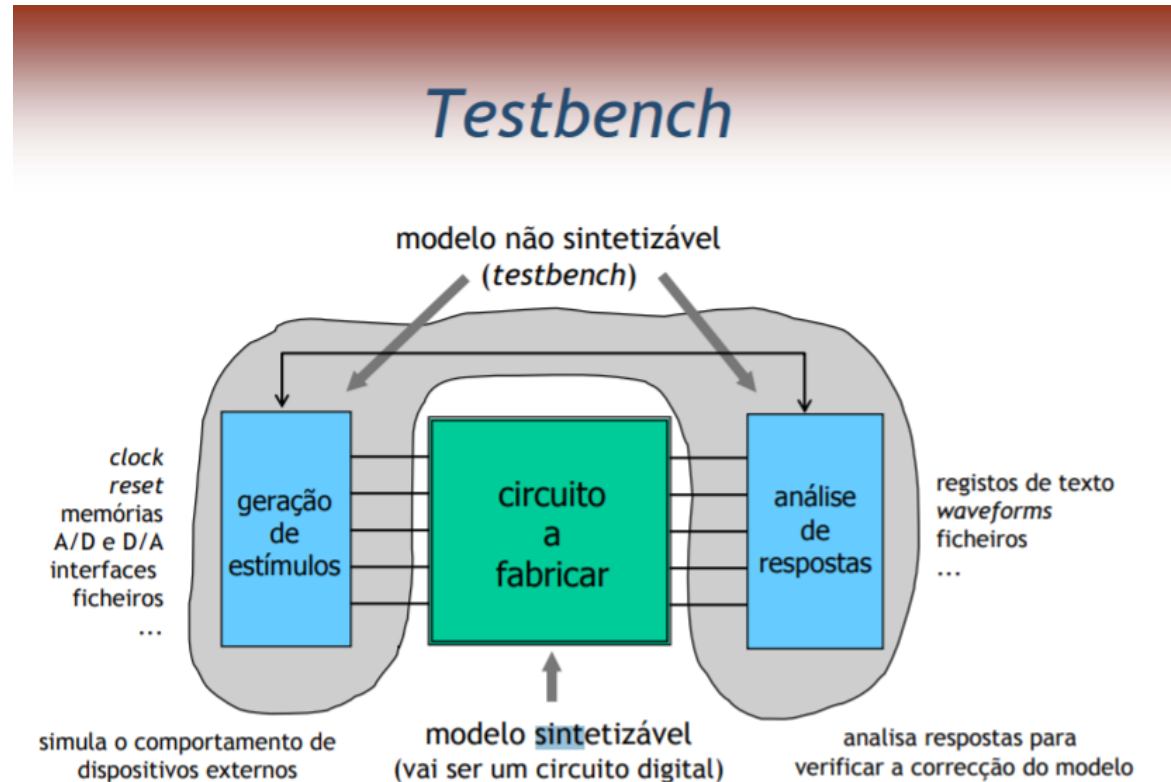


```
integer i; // declare the index for the FOR LOOP

always @(inp or cnt)
begin
    result[7:4] = 0;
    result[3:0] = inp;
    if (cnt == 1)
        begin
            for (i = 4; i <= 7; i = i + 1)
                begin
                    result[i] = result[i-4];
                end
            result[3:0] = 0;
        end
    end
```

Importante

- Modelos não sintetizáveis
 - Não são traduzidos para hardware
 - Definição de estímulos para simulação



FIM